ENGINEERING PROMPT PLAYBOOK

30 goal prompts

Decentralized short-video network

with a custom application blockchain

CODEX CLOUD + GITHUB

Copy-ready goals, model and effort assignments, implementation constraints, acceptance tests and evidence-based handoffs.

30	12	18
BOUNDED GOALS	ASTRA GOALS	SOL GOALS

Scope: permissioned web beta, community-operated video hosting, and a real Cosmos SDK / CometBFT chain. No token launch or production-security promise.

Version 1.0 | 13 October 2026

Companion to the project blueprint

START HERE

How to use this playbook

Use one goal at a time. These are 30 work packages, not a promise of completion after 30 messages. Corrections, reviews and blocked tests may require additional runs. No implementation or benchmark has been executed by this prompt pack.

Prepare the repository once

Place the companion pack at the repository root. It contains AGENTS.md, docs/goals/01-...md through 30-...md, a progress tracker, a text reference and the original blueprint PDF. Merge with existing instructions rather than blindly overwriting them. The pack contains prompts and documentation, not a working app.

Run a bounded goal

Select its model and effort in your Codex client, verify the target repository and branch, then paste the goal or ask Codex to execute its file. Use the common instructions on page 4 as AGENTS.md. Repository instruction files are supported by Codex [O04]. Instructions naming a model do not change the selected model automatically.

Start dependent work from the reviewed base commit. Keep failed work on its branch, inspect the diff and tests, then approve the merge yourself. Store the tested code revision, commands, logs and blockers in docs/evidence/goal-NN.md. A new task must read this evidence instead of relying on chat memory.

Model and effort assignments

Suggested allocation: Astra Extra High for Goals 01-02 and 29-30; Astra High for 05-12; GPT-6.1 Sol Medium for 03-04; GPT-6.1 Sol High for 13-28. This is a proposed engineering allocation, not a measured optimum.

OpenAI documents Astra and GPT-6.1 Sol, and High/Extra High effort for difficult work. Availability and controls depend on account, client and rollout [O03]. Choose the available setting explicitly and record substitutions; do not pretend an unavailable model was used. Extra effort does not certify correctness.

Scope boundary

Target the blueprint's permissioned community beta: web application, real chain, designated hosts, HTTPS delivery, moderation, recovery and operator tools. Native mobile apps, permissionless admission, tradable tokens, optional P2P and advanced cryptographic storage proofs remain separate extensions [B01, pp. 3-5, 15].

> A goal is verified only when its required checks actually ran against the reported revision. IMPLEMENTED, VERIFIED, BLOCKED and DEFERRED are different statuses. Never convert a missing test environment into a pass.

The source register is on page 39. Blueprint page references appear on every goal. All imperative requirements in this playbook are proposed implementation instructions.

REMOTE DEVELOPMENT

Codex Cloud + GitHub setup

Use GitHub for source and review; use Codex Cloud for development tasks; use an approved integration runner for tests that exceed the workspace; deploy the eventual network to separate operator-controlled machines. This separation is the recommended project workflow.

Connect and prepare

In the web or desktop app, create a cloud environment, select the GitHub repository, review the prepared setup and publish it. Codex Cloud supports coding tasks and review of changes or pull requests [O01]. Create or edit the setup before sending implementation work that needs new dependencies.

The current environment guide documents install/start configuration and isolated task workspaces. Saved task state does not replace Git. It also lists computer/browser use as unsupported; do not assume interactive browser tools exist. Verify headless tests separately [O02].

Use two test paths

Run permitted builds, unit tests and small integration checks inside the task. Put heavier four-validator, media, browser and fault/load checks on a validated Linux CI/test runner when necessary. GitHub Actions can run project test commands on hosted runners [G01]. No claim is made that the entire stack fits your current cloud workspace.

Keep spending, external access and deployment under your control. Use only development identities. Production validators, retained videos and the public service are not the prepared agent workspace.

ENVIRONMENT SETUP REQUEST / PART OF GOAL 04

Prepare this repository for Codex Cloud development, not production hosting. Read AGENTS.md and the current dependency decisions. Inspect the actual environment and test Go, Python, Node, FFmpeg, PostgreSQL, RabbitMQ, permitted local ports and any available container runtime. Do not assume Docker or browser tools exist.

Create repeatable install/start commands and smoke checks. Use only synthetic data and freshly generated development credentials. Keep private keys out of Git and the reusable published template. Limit network access to approved package/documentation endpoints and development services.

Record observed capabilities in docs/environment-capabilities.md. Separate checks that run here from full devnet, browser, failure and load tests requiring a different authorized runner. If a check cannot run, explain the blocker and preserve its exact command; never replace it with mocked success. Ask before changing permissions, spending money or provisioning infrastructure.

Report configuration changes and actual test results. Publishing or republishing the prepared environment is a user action unless explicitly delegated. Recheck the branch and base commit in the next task. This setup request belongs to Goal 04, not an additional implementation goal.

Current-versus-legacy note: follow the environment controls actually shown in your account. This guide uses OpenAI's current Cloud environments page, not the older setup-only secrets/container-caching instructions [O02].

PERSISTENT PROJECT INSTRUCTIONS

Common contract: AGENTS.md

Copy this contract into the root AGENTS.md before running the goals. Preserve stricter existing project instructions. Each goal relies on this contract rather than repeating it in full.

Scope and authority

Build the permissioned community beta described in docs/blueprint.md. The original PDF remains the source for ambiguous extraction. Use a real Cosmos SDK / CometBFT application chain, not a simulated ledger. Keep media, personal data and ordinary social interactions off-chain. No tradable token, new consensus algorithm, proprietary-code copying, mandatory commercial cloud, or unreviewed cryptography. Treat specifications and numeric targets as proposals until approved; never claim an audit or production readiness.

Context and changes

Before a task, read this AGENTS.md, docs/progress.md, approved decisions and only the relevant specification sections. Verify the repository, branch and base commit; preserve unrelated work. Implement the current goal in small reviewable changes. Do not merge, force-push, publish releases, buy services, change access, or deploy publicly without explicit approval. Identify security-critical ambiguity and request a decision; document reversible implementation choices without presenting them as user approval.

Cloud and credentials

Inspect the actual runtime and network permissions. Never assume Docker, browser automation, extra disk, external peer ports or a running service exists. Do not bypass sandbox or network restrictions. Use synthetic fixtures and fresh development-only keys; never request live validator, governance or production signing keys. Keep secrets, key files, media uploads and database dumps out of Git, logs and shared environment templates. Prepare reproducible external test instructions when the workspace cannot run a required check.

Implementation discipline

Pin compatible dependency versions and verify APIs against official documentation. Use typed interfaces, explicit error handling, bounded work, object authorization and retry-safe state transitions. Chain execution must be deterministic and free of network calls or local-clock decisions. Provider signatures are not interchangeable with service signatures. A broadcast acknowledgement is not committed execution success. Mocking is allowed in unit tests only; never use it as end-to-end evidence.

Verification and status

Test every change, including relevant negative cases. Never delete or weaken a requirement, skip a failing check silently, replace an implementation with a stub, or invent logs to obtain a pass. Record the commit, runtime, commands, exit codes, test counts and artifact locations. Distinguish IMPLEMENTED, VERIFIED, BLOCKED and DEFERRED. A skipped or unavailable required test is not passed. Keep immutable evidence for the tested code revision and rerun affected checks after fixes.

Handoff

Update docs/progress.md and docs/evidence/goal-NN.md. Return changed files, verified outcomes, failures, unrun tests, residual risks and the next safe action. Maintain the requirements-to-tests matrix. End with a proposed pull-request title and concise review summary; create the PR only when authorized. Do not declare the whole project finished because the current goal is done.

Project commands and schemas referenced in the goals are interfaces to implement and test, not commands asserted to exist already. Empty directories, generated stubs and mocked services are never end-to-end verification.

EXECUTION MAP

Goals 01-15: foundations and media

Each row links to a copy-ready goal. Read prerequisites on the goal page. Assignments are recommendations; use only settings available in your client [O03].

GOAL	DELIVERABLE	MODEL	EFFORT	PAGE
01	Requirements, scope and threat model	Astra	Extra High	7
02	Protocol schemas and invariants	Astra	Extra High	8
03	Monorepo, toolchains and CI	6.1 Sol	Medium	9
04	Codex Cloud development environment	6.1 Sol	Medium	10
05	Chain application, genesis and devnet	Astra	High	11
06	Authority and validator lifecycle	Astra	High	12
07	Provider and content registries	Astra	High	13
08	Lease lifecycle and capacity accounting	Astra	High	14
09	Beta attestations and audit evidence	Astra	High	15
10	Policy governance and chain upgrades	Astra	High	16
11	Transaction broadcaster and outbox	Astra	High	17
12	Committed-state indexer and reconciliation	Astra	High	18
13	Authenticated uploads and quarantine	6.1 Sol	High	19
14	Isolated video processing workers	6.1 Sol	High	20
15	Media manifests and HTTPS playback	6.1 Sol	High	21

Gate A: approved scope and protocol (01-02). Gate B: real chain and committed-state integration (05-12). Scaffolding and an untested CI file do not meet either gate.

Foundation: Goals 03-04 prepare the repository and execution environments. Goal 13 includes a minimal authenticated boundary; Goal 19 completes enrollment, recovery and portability. Do not expose an unsecured intermediate build.

EXECUTION MAP

Goals 16-30: hosting to release

Each row links to a copy-ready goal. Read prerequisites on the goal page. Assignments are recommendations; use only settings available in your client [O03].

GOAL	DELIVERABLE	MODEL	EFFORT	PAGE
16	Storage-provider agent	6.1 Sol	High	22
17	Replication, monitoring and repair	6.1 Sol	High	23
18	End-to-end network milestone	6.1 Sol	High	24
19	Accounts, recovery and data portability	6.1 Sol	High	25
20	Viewer web application	6.1 Sol	High	26
21	Creator publishing interface	6.1 Sol	High	27
22	Social features, search and feeds	6.1 Sol	High	28
23	Moderation, blocking and removal	6.1 Sol	High	29
24	Operator dashboard and node onboarding	6.1 Sol	High	30
25	Observability, quotas and usage controls	6.1 Sol	High	31
26	Backups, restoration and signer safety	6.1 Sol	High	32
27	Packaging, installation and release preparation	6.1 Sol	High	33
28	Performance, load and capacity evidence	6.1 Sol	High	34
29	Adversarial review and fault testing	Astra	Extra High	35
30	Final acceptance and independent installation	Astra	Extra High	36

Gate C: upload, replication, final chain commitment and real playback survive origin loss (18). Gate D: account and safety workflows work (19-24). Gate E: evidence and human approvals justify the release decision (25-30).

If a prerequisite is blocked, record it. Independent work may proceed only when its own dependencies are satisfied; never mark the blocked prerequisite complete. Optional continuation, repair and review prompts are on page 37.

GOAL 01 / 30

Requirements, scope and threat model

GPT-6 Astra	EFFORT: EXTRA HIGH

Prerequisites: None; the blueprint must be available in the repository.

Blueprint reference: pp. 3-6, 19-21, 25-28 [B01]. Settings are selected in the client.

COPY PROMPT

Complete Goal 01. Read AGENTS.md, docs/progress.md and the approved specifications for this goal. Verify the target branch, base commit and prerequisite evidence. Execute only this goal; apply the common contract.

Convert the blueprint into an actionable, traceable beta specification before implementing application features.

IMPLEMENTATION REQUIREMENTS

1. Inventory the repository and distinguish existing working code, placeholders and missing components. Create docs/requirements.md with stable requirement IDs, a requirements-to-tests matrix, docs/threat-model.md and a decision register. Link each requirement to its blueprint page and identify its accountable role.
2. Define the beta: responsive web app; invited accounts; four equal-power development validators; designated storage hosts; HTTPS delivery; quotas; signed beta attestations; no monetary rewards. Separate user login, social records, media transport, storage commitments, governance and consensus trust boundaries.
3. Record unresolved decisions about custody/recovery, operator admission, moderation/removal, retention, sponsorship, licensing and audience policy. Request approval only where the answer changes security or public-operation commitments. Mark all suggested defaults as proposed, and keep public launch disabled.
4. Threat-model hostile uploads, unauthorized transactions, compromised keys, dishonest providers/auditors, unavailable replicas, chain stalls, stale gateways, abusive content and supply-chain attacks. Assign a mitigation, a test and a residual limitation to each. Include the distinction between an independent process and an independent operator.
5. Create docs/progress.md for all 30 goals. Define evidence statuses and dependencies, and identify tests needing cloud commands, a fuller Linux test runner, or independent machines. Do not promise dates, token totals, market adoption or production security.

ACCEPTANCE CHECKS

Check that every in-scope blueprint requirement has an owner, implementation goal and test or explicit decision blocker. Resolve contradictions without silently expanding the release.

Walk through upload-to-playback, origin loss, provider departure, chain stall and removal as specification scenarios. Record missing decisions; a polished document with unresolved critical rules is not an approved protocol.

REQUIRED HANDOFF

Produce the specification files, decision questions and a proposed Goal 02 input checklist. No application code is required for this goal.

GOAL 02 / 30

Protocol schemas and invariants

GPT-6 Astra	EFFORT: EXTRA HIGH

Prerequisites: Goal 01; security-critical design decisions approved or explicitly blocked.

Blueprint reference: pp. 8-17, 26-27 [B01]. Settings are selected in the client.

COPY PROMPT

Complete Goal 02. Read AGENTS.md, docs/progress.md and the approved specifications for this goal. Verify the target branch, base commit and prerequisite evidence. Execute only this goal; apply the common contract.

Define exact versioned contracts shared by the blockchain, services, provider agent and web client.

IMPLEMENTATION REQUIREMENTS

1. Write docs/protocol.md and machine-readable draft schemas: versioned Protobuf state, transaction, query and genesis types; API contracts; media manifests; provider capability manifests; and beta evidence envelopes. Specify IDs, field types, maximum sizes, integer ranges, units, enum handling, digest algorithms, signature domains and error codes.
2. Define provider, content, lease, authority, policy and evidence state machines. Separate a committed hosting obligation from observed reachability and from community publication approval. State who signs each transition, who may relay it, how replay/idempotency works, and when declared capacity is reserved or released.
3. Specify bounded expiry indexes and deterministic execution. Define committed chain time versus real-time host deadlines, pending requests during stalls, and treatment of expired offers or stale evidence. No handler may fetch external data or decide consensus state from a local clock.
4. Choose vetted serialization/canonicalization for signed off-chain records and publish valid/invalid cross-language vectors. Reuse the SDK transaction envelope. Define version negotiation, key rotation, removal tombstones, privacy classifications and data retained for reconciliation.
5. Create executable schema/fixture checks where tools permit. Keep protocol invariants distinct from implementation details. Explicitly label signed receipts and audits as attestations, not proof of independent storage or future delivery.

ACCEPTANCE CHECKS

Validate boundary values, malformed/unknown fields, duplicate IDs, wrong domains, expired terms and unauthorized signers against the proposed contracts. Confirm Go/Python/TypeScript can represent the same values without precision loss.

Review each transition for underflow/overflow, stale versions, retries, withdrawal and conflicting governance updates. The specification must support the exact failure tests in blueprint page 26.

REQUIRED HANDOFF

Deliver the contracts, fixture suite and unresolved decisions. Freeze approved protocol v0 interfaces for Goals 03-18; require a recorded decision for incompatible changes.

GOAL 03 / 30

Monorepo, toolchains and CI

GPT-6.1 Sol	EFFORT: MEDIUM

Prerequisites: Goals 01-02; selected contracts and dependency decisions.

Blueprint reference: pp. 7, 19, 25, 27 [B01]. Settings are selected in the client.

COPY PROMPT

Complete Goal 03. Read AGENTS.md, docs/progress.md and the approved specifications for this goal. Verify the target branch, base commit and prerequisite evidence. Execute only this goal; apply the common contract.

Create a buildable repository foundation with reproducible dependency management and truthful continuous-integration results.

IMPLEMENTATION REQUIREMENTS

1. Use the blueprint monorepo layout for apps/web, services/api, services/indexer, services/broadcaster, chain, proto, agents/provider, workers/media, packages/protocol, deployments, tests and docs. Preserve any useful existing files; do not generate fake finished services.
2. Verify a supported compatible Cosmos SDK / CometBFT pair and the required Go toolchain from official sources. Pin Python, Node, package managers, code generators and runtime dependencies with lockfiles or equivalent. Record versions, source URLs, licences and compatibility rationale in docs/dependencies.md; do not silently select commercial-only modules.
3. Create format, lint, type-check, schema-generation and unit-test entry points. Add GitHub Actions jobs with minimal permissions, bounded runtime and isolated fixtures. Pin action dependencies using the supported locking mechanism or verified full commit SHAs. Do not invent SHA values.
4. Protect CI from untrusted pull-request input and secret exposure. No privileged checkout of untrusted code, production access, automatic merging or deployment. Add a PR template that records requirement IDs, actual test evidence and outstanding risks; add a proposed ownership/review policy without inventing reviewers.
5. Provide README installation instructions and .env.example with placeholders only. Exclude generated private material, dependency caches, build outputs and uploaded media from Git. An empty test suite must not report project verification.

ACCEPTANCE CHECKS

From a clean checkout, install pinned dependencies, generate contracts and run the available checks. Test that a deliberately failing fixture makes the relevant job fail, then remove that deliberate failure.

Inspect workflow permissions, dependency references, ignore rules and generated diffs. Report network/toolchain blockers rather than pretending CI ran remotely.

REQUIRED HANDOFF

Return the working skeleton, exact commands and CI configuration. Record what is scaffolding versus implemented; hand off environment needs to Goal 04.

GOAL 04 / 30

Codex Cloud development environment

GPT-6.1 Sol	EFFORT: MEDIUM

Prerequisites: Goal 03; access to the intended GitHub repository and cloud environment.

Blueprint reference: pp. 7, 11-13, 22, 27 [B01]. Settings are selected in the client.

COPY PROMPT

Complete Goal 04. Read AGENTS.md, docs/progress.md and the approved specifications for this goal. Verify the target branch, base commit and prerequisite evidence. Execute only this goal; apply the common contract.

Prepare a tested cloud development setup and a separate full-integration profile, without assuming the workspace is a production host.

IMPLEMENTATION REQUIREMENTS

1. Inspect actual CPU, memory, free disk, OS, runtimes, process/port support, container runtime access and network restrictions. Test required package downloads and local service connections. Write docs/environment-capabilities.md with observed results, not a vendor-feature assumption.
2. Create repeatable install and startup commands for the pinned Go/Python/Node tools, PostgreSQL, RabbitMQ, FFmpeg and development storage. Integrate them into the available Codex Cloud setup fields. Make startup idempotent, check readiness, report failures, and avoid regenerating existing application or chain state.
3. Provide a minimal cloud profile and a full Docker Compose or equivalent Linux test profile. If containers are unavailable, start permitted local service processes or mark dependent tests blocked; do not emulate a real database with an incompatible substitute and call it verified.
4. Use loopback/private bindings, disposable accounts, separate service identities and fresh test-only credentials. Generate future devnet keys per test workspace, outside Git and the shared prepared template. Never clone an active validator identity into concurrent tasks.
5. Create project commands for environment diagnosis, service start/stop, clean test reset and smoke tests. Destructive resets must target a named disposable workspace and require explicit confirmation. Document how to republish dependency changes and verify the checked-out branch/base commit before the next goal.

ACCEPTANCE CHECKS

Start from a clean permitted runtime and demonstrate database queries, a real queue round trip and a small FFmpeg operation. Restart without losing expected state; prove absent credentials or unavailable ports fail clearly.

Run the cloud checks and report which full-stack, browser, network-fault or load tests require another runner. If those facilities are missing, preserve reproducible commands and a BLOCKED status rather than weakening acceptance.

REQUIRED HANDOFF

Deliver setup/start instructions, environment evidence, resource observations and the full-runner profile. Do not provision paid machines or change external permissions without approval.

GOAL 05 / 30

Chain application, genesis and devnet

GPT-6 Astra	EFFORT: HIGH

Prerequisites: Goals 02-04; compatible framework/toolchain confirmed.

Blueprint reference: pp. 8, 11-12, 26 [B01]. Settings are selected in the client.

COPY PROMPT

Complete Goal 05. Read AGENTS.md, docs/progress.md and the approved specifications for this goal. Verify the target branch, base commit and prerequisite evidence. Execute only this goal; apply the common contract.

Create the real Go application-chain foundation and demonstrate four validator processes committing the same chain state.

IMPLEMENTATION REQUIREMENTS

1. Inspect the official scaffold for the pinned SDK and CometBFT versions. Create the chain binary, application wiring, address configuration, service routes and module registration. Remove unused staking/EVM/bridge/token functionality deliberately; do not hide a default economic model behind a renamed chain.
2. Implement a validated genesis schema for chain ID, timestamp, protocol version, initial authorities, public validator keys, equal voting powers, admitted application accounts and resource bounds. Keep the initial validator set static until Goal 06 adds its complete update controls. State that this is scaffolding, not all project modules.
3. Use existing signature/account-sequence mechanisms with size and gas limits. Reject unknown or unauthorized application messages. Test initialization ordering and genesis import/export. Prevent two independent module paths from setting validator power.
4. Add a devnet launcher for four distinct data directories, peer identities and consensus keys. Use a unique disposable chain identity per independent test run, distribute one validated genesis, and keep keys out of version control and shared environment templates.
5. Provide status, logs and safe stop/resume commands. Include ordinary process-based execution where permitted and a full Linux/container profile. Do not reset genesis or manufacture block history to recover from an error.

ACCEPTANCE CHECKS

From a clean workspace, start all four validators and compare committed application hashes at the same height. Restart and confirm continued progression with preserved state.

Stop one validator, then two, and observe continued commits versus stalled new commits under the declared test assumptions. Validate recovery and reject malformed genesis, duplicate keys and inconsistent chain IDs.

If the current runtime cannot start the real network, run available unit tests and leave devnet verification blocked for the named runner.

REQUIRED HANDOFF

Record binary/tool versions, public genesis digest, test topology and logs. Leave application-module completion for Goals 06-10; never present a mock ledger as this milestone.

GOAL 06 / 30

Authority and validator lifecycle

GPT-6 Astra	EFFORT: HIGH

Prerequisites: Goal 05 and the approved authorization/governance contracts.

Blueprint reference: pp. 8-12, 19, 21 [B01]. Settings are selected in the client.

COPY PROMPT

Complete Goal 06. Read AGENTS.md, docs/progress.md and the approved specifications for this goal. Verify the target branch, base commit and prerequisite evidence. Execute only this goal; apply the common contract.

Implement x/authority as the sole source of permissioned validator membership and authorized chain-write roles.

IMPLEMENTATION REQUIREMENTS

1. Implement typed council seats, submitter roles, provider/service admission, proposal IDs, unique approvals, expiry and scheduled validator changes. Use the approved threshold/delay rules; implement the minimal proposal machinery needed here, leaving broader policy/upgrades to Goal 10.
2. Connect updates to the exact initialization and block lifecycle of the selected SDK/CometBFT release. Specify and test when a proposed public key or power change becomes effective. No staking or other module may overwrite the resulting set.
3. Enforce authorization, signature verification, replay/sequence rules, transaction size, gas and deterministic quotas in committed execution as well as admission checks. A permissive mempool must not bypass consensus-critical validation.
4. Validate resulting sets for duplicate keys, invalid powers, overflow, removal of all voting power and governance-policy violations. Test consensus-key rotation separately from provider, service and council-key rotation.
5. Document ordinary admission/removal, emergency local endpoint disablement and loss-of-quorum recovery. Keep governance approvals distinct from consensus votes. No administrator endpoint may fabricate a committed decision or override finality.

ACCEPTANCE CHECKS

Exercise approved and rejected proposals, repeated votes, stale approvals, unauthorized submitters, wrong signatures and expired execution. Confirm no state mutation from rejected messages.

On the real devnet, add/rotate/remove a test validator and compare the effective set at the prescribed heights. Verify one-third/quorum behaviour for the resulting voting powers, not just node count.

Fuzz bounded parser/state-transition inputs and compare deterministic replay at a fixed committed height.

REQUIRED HANDOFF

Deliver authority code, lifecycle fixtures and a reviewer-focused authorization map. Flag any framework-lifecycle or licensing uncertainty before building dependent modules.

GOAL 07 / 30

Provider and content registries

GPT-6 Astra	EFFORT: HIGH

Prerequisites: Goals 02, 05-06; signed manifest contracts available.

Blueprint reference: pp. 9-11, 14, 27 [B01]. Settings are selected in the client.

COPY PROMPT

Complete Goal 07. Read AGENTS.md, docs/progress.md and the approved specifications for this goal. Verify the target branch, base commit and prerequisite evidence. Execute only this goal; apply the common contract.

Implement x/providers and x/content with minimal public state and precise ownership, versioning and admission rules.

IMPLEMENTATION REQUIREMENTS

1. Create Protobuf types, keepers, message/query handlers, genesis validation, events and migrations for both modules. Providers expose stable IDs, signing-key references, capabilities, status, admitted operator-group metadata and a digest of a signed off-chain endpoint manifest.
2. Content records contain an opaque ID, immutable package/manifest version, digest, byte count, approved submitting host and coordination status. Keep video bytes, user secrets, personal account fields and moderation narratives off-chain. Explain that opaque hashes can still be linkable.
3. Enforce eligible submitters, object-level ownership, validated digest lengths/algorithms, bounded strings and byte totals, unique IDs/versions and typed errors. Separate provider key updates from consensus-key changes. Define what disablement means for existing obligations without deleting history.
4. Implement content withdrawal so new sponsored commitments cannot be created or silently revived. Allow a new encoding only as a new package version. Do not treat a registry entry as copyright proof, a hosting receipt, or publication approval.
5. Provide paginated queries and stable events for the later indexer. Endpoint URLs stay in off-chain manifests where possible; validate the metadata now and leave actual fetches to services, never consensus execution.

ACCEPTANCE CHECKS

Commit valid registration/update/withdrawal transactions on the devnet. Reject cross-owner edits, unauthorized provider updates, duplicate versions, invalid digests, oversized metadata and replayed requests.

Round-trip genesis and reconstruct query results from committed state. Compare replay hashes and test pagination boundaries and event schemas.

Assert that no private user details or media payloads appear in stored state or event fixtures.

REQUIRED HANDOFF

Provide a minimal real-transaction demo and negative-test evidence. Hand off immutable package identity and provider-admission APIs to the lease module.

GOAL 08 / 30

Lease lifecycle and capacity accounting

GPT-6 Astra	EFFORT: HIGH

Prerequisites: Goal 07 and approved lease state/terms specification.

Blueprint reference: pp. 9-10, 13-15, 26 [B01]. Settings are selected in the client.

COPY PROMPT

Complete Goal 08. Read AGENTS.md, docs/progress.md and the approved specifications for this goal. Verify the target branch, base commit and prerequisite evidence. Execute only this goal; apply the common contract.

Implement x/leases so each storage obligation has authorized parties, immutable terms, bounded accounting and correct expiry semantics.

IMPLEMENTATION REQUIREMENTS

1. Implement offer, acceptance, renewal and end transitions using stable offer/lease/business IDs. Bind acceptance to provider signature, exact content version, manifest digest, bytes, terms version and real-time deadline. A submitting service cannot sign acceptance on a provider's behalf.
2. Reserve and release declared capacity atomically according to the approved lifecycle. Prevent duplicate provider/content-version copies from counting as separate replicas. Enforce per-provider and global bounds, nonnegative counters, valid terms and idempotent business operations across distinct transaction hashes.
3. Define ACTIVE as a committed obligation, not automatic evidence of retrieval, physical independence or publication. Keep observed AT_RISK availability off-chain unless an explicitly approved attestation transition applies. Renewal cannot silently change the committed bytes.
4. Implement deadline indexes and bounded expiry processing. Use committed time/context for consensus changes; document that host obligations have real-time deadlines even when the chain stops. Expired offers/evidence cannot renew current status.
5. Integrate provider disablement, content withdrawal, cancellation, repair handoff and restart behaviour. Preserve history needed for audit and reconciliation; never release the same reservation twice.

ACCEPTANCE CHECKS

Property-test transition sequences with an independent accounting oracle. Cover over-capacity acceptance, stale terms, wrong provider keys, duplicate IDs, overflow, repeated ending, early/late renewal and withdrawn content.

Run real transactions with timeout/retry scenarios and demonstrate one final obligation with correct reserved bytes. Confirm failed operations leave consistent state.

Replay the same committed inputs and compare hashes. Test a large expiring catalogue to show bounded per-block work rather than an unbounded sweep.

REQUIRED HANDOFF

Document invariants, storage indexes, event payloads and exhaustive negative cases. Provide the evidence required before scheduler and broadcaster integration.

GOAL 09 / 30

Beta attestations and audit evidence

GPT-6 Astra	EFFORT: HIGH

Prerequisites: Goal 08; evidence trust model approved in Goal 02.

Blueprint reference: pp. 9-10, 15, 19, 24 [B01]. Settings are selected in the client.

COPY PROMPT

Complete Goal 09. Read AGENTS.md, docs/progress.md and the approved specifications for this goal. Verify the target branch, base commit and prerequisite evidence. Execute only this goal; apply the common contract.

Implement x/evidence for signed beta attestations without claiming a trustless proof-of-storage system.

IMPLEMENTATION REQUIREMENTS

1. Implement authorized auditors, schema versions, batch IDs, epochs/rounds, signer references, evidence digests and bounded result records. Keep full observation payloads off-chain under the approved retention policy, with retrieval/checksum verification for reviewers.
2. Validate signatures, allowed signer roles, correct round, unique evidence IDs, lease/content binding, digest formats, field bounds and maximum batch size. Reject replay, stale evidence and unknown schemas. Define exactly which status may change after acceptance; evidence cannot extend a lease by implication.
3. Create a minimal off-chain audit producer for permitted test endpoints: retrieve sampled segments or whole packages, verify digests and record timestamps, results and failure reasons. Use the network restrictions defined in the provider contract; do not fetch endpoints from chain handlers.
4. Display evidence mode explicitly as provider receipt or auditor attestation. Record limitations: auditors can lie, providers may fetch from another host, and node IDs do not prove independent replicas.
5. Defer financial rewards, automatic slashing, randomized cryptographic proofs and Filecoin integration. Do not introduce a home-made Merkle scheme and call it a uniquely stored replica proof.

ACCEPTANCE CHECKS

Accept a legitimate bounded audit batch and reject wrong signers, altered observations, duplicate evidence, expired rounds and oversized batches. Test evidence availability failure separately from chain inclusion.

Simulate a dishonest but admitted auditor and document that valid signing does not make its observation truthful. A stale or passing audit must not revive expired/withdrawn hosting.

Measure transaction bytes, execution work and state growth on the declared fixtures without extrapolating a network throughput claim.

REQUIRED HANDOFF

Deliver module tests, audit fixtures and accurate UI/API evidence labels. Record a separate future cryptographic-verification decision gate, not an unfinished feature disguised as complete.

GOAL 10 / 30

Policy governance and chain upgrades

GPT-6 Astra	EFFORT: HIGH

Prerequisites: Goals 06-09; migration and governance specifications approved.

Blueprint reference: pp. 8-12, 21-22, 26 [B01]. Settings are selected in the client.

COPY PROMPT

Complete Goal 10. Read AGENTS.md, docs/progress.md and the approved specifications for this goal. Verify the target branch, base commit and prerequisite evidence. Execute only this goal; apply the common contract.

Complete x/policy and the governed change/upgrade path while preserving the established authority and accounting invariants.

IMPLEMENTATION REQUIREMENTS

1. Extend the authority proposal primitives with typed parameter changes, admission limits, policy versions and scheduled activation. Validate unique approvals, threshold, expiry and review delay. Protect transaction sizes, gas, quotas, lease bounds and resulting validator policies against unsafe updates.
2. Implement the approved chain-upgrade mechanism using supported framework components. Specify binary/version compatibility, module migration versions, activation point, preconditions and failure behaviour. Do not add commercial-only components without explicit licensing approval.
3. Create a rehearsal harness from representative devnet state containing providers, active/expired leases, evidence and pending proposals. Compare pre/post invariants and verify application/indexer schema compatibility.
4. Write runbooks for preparing a release, reviewing source/digests, operator installation, planned halt/restart and failed upgrades. Explain when rollback is unsafe after state migration. A governance stall cannot be fixed by an unaudited finality bypass.
5. Separate local emergency controls, protocol changes and public publication authority. Do not auto-approve policies, tag releases, rotate live keys or change real operators as part of this task.

ACCEPTANCE CHECKS

Test duplicate votes, removed council seats, invalid bounds, concurrent proposals, activation timing and insufficient approval. Assert unauthorized policy writes fail at execution.

Rehearse a successful migration and a deliberately incompatible binary on disposable state; verify controlled failure without resetting history. Check exports, restart and replay at the specified heights.

Confirm the same tested chain revision still passes the authority, lease and evidence suites.

REQUIRED HANDOFF

Return governed change fixtures, migration evidence and operator instructions. List every step that still needs a human approval or independent security review.

GOAL 11 / 30

Transaction broadcaster and outbox

GPT-6 Astra	EFFORT: HIGH

Prerequisites: Goals 04, 07-10; approved service/provider signing boundaries.

Blueprint reference: pp. 8, 10, 13, 26-27 [B01]. Settings are selected in the client.

COPY PROMPT

Complete Goal 11. Read AGENTS.md, docs/progress.md and the approved specifications for this goal. Verify the target branch, base commit and prerequisite evidence. Execute only this goal; apply the common contract.

Build a retry-safe bridge from application intent to committed chain execution without confusing submission with success.

IMPLEMENTATION REQUIREMENTS

1. Implement services/broadcaster with typed requests and a PostgreSQL transactional outbox. Persist intent IDs, authorized request context, signer identity, attempts, transaction hashes, expiry, results and reconciliation state. Couple application intent and outbox insertion in one database transaction.
2. Use approved service keys only for their permitted messages. Providers sign their own acceptances; support forwarding their valid signed transactions or the approved provider-side submission path. Never synthesize provider consent from a service credential.
3. Handle sequence allocation/concurrency per signer, bounded retries, backoff, transaction deadlines and uncertain submission. Reconcile known transaction/business IDs before resending; distinguish transport error, mempool rejection, failed execution and successful commitment.
4. Expose requested, submitted, committed, failed and uncertain/pending states to callers. Authenticate internal endpoints and enforce object permissions. A chain outage queues bounded expiring work but cannot create final leases in the database.
5. Keep sensitive keys behind the approved development signing boundary, redact logs and make gateway/chain ID configuration explicit. No undocumented network dependency or live credential is required for the devnet.

ACCEPTANCE CHECKS

Against the real devnet, inject timeouts before/after submission, worker death after commit, duplicate requests and concurrent same-signer requests. Assert one business effect, no leaked capacity and eventual accurate status.

Reject unauthorized messages and attempted provider impersonation. Stop quorum and verify the API remains pending or fails honestly rather than returning committed success.

Restart PostgreSQL/broadcaster and reconcile from persisted state without manual history edits.

REQUIRED HANDOFF

Deliver exact failure traces, outbox schema, service API and signing-boundary documentation. Give the indexer stable identities and canonical commitment evidence.

GOAL 12 / 30

Committed-state indexer and reconciliation

GPT-6 Astra	EFFORT: HIGH

Prerequisites: Goal 11; real chain query/event contracts and PostgreSQL.

Blueprint reference: pp. 9-10, 12-13, 26 [B01]. Settings are selected in the client.

COPY PROMPT

Complete Goal 12. Read AGENTS.md, docs/progress.md and the approved specifications for this goal. Verify the target branch, base commit and prerequisite evidence. Execute only this goal; apply the common contract.

Build restartable off-chain projections from committed chain state, with explicit staleness and recoverable event gaps.

IMPLEMENTATION REQUIREMENTS

1. Implement services/indexer using committed blocks/events and supported chain queries. Version event decoding. Include chain identity, block height and stable transaction/message/event identity in deduplication keys; never index an uncommitted submission as final.
2. Write projection updates and checkpoint advancement atomically. Handle repeated delivery, restart, partial batches and out-of-order inputs safely. Preserve the canonical event/source reference for each important state change.
3. Implement backfill and periodic reconciliation of providers, content, active leases, deadlines and policy versions. Specify archival/pruning requirements and behaviour when history is unavailable. Never interpret a missing range as an empty valid history.
4. Expose indexed height, canonical height/commit age, last reconciliation and stale status. Treat the database as a read model, not a replacement chain. Disclose trust in the configured read gateway; comparing two unauthenticated responses is not light-client verification.
5. Add a clean rebuild command for disposable projections, guarded against deleting authoritative application data. Keep private social records separate from rebuildable public coordination data.

ACCEPTANCE CHECKS

Kill the indexer between event handling and checkpoint persistence, resume and prove exactly one projection effect. Rebuild from the same chain and compare canonical rows after normalization.

Inject malformed event versions, forged responses, missing blocks and chain-ID mismatch; reject or block with clear diagnostics. Verify reconciliation detects deliberately altered projections.

Stop/resume consensus and show stale read status without falsifying a lease renewal or publication approval.

REQUIRED HANDOFF

Return checkpoint/replay evidence, source-trust documentation and read APIs for hosting services. Record archive requirements and any blocked full-history tests.

GOAL 13 / 30

Authenticated uploads and quarantine

GPT-6.1 Sol	EFFORT: HIGH

Prerequisites: Goals 02, 04, 11-12; minimum account/permission contract approved.

Blueprint reference: pp. 5, 13, 16-17, 19 [B01]. Settings are selected in the client.

COPY PROMPT

Complete Goal 13. Read AGENTS.md, docs/progress.md and the approved specifications for this goal. Verify the target branch, base commit and prerequisite evidence. Execute only this goal; apply the common contract.

Implement a real bounded upload service that cannot publish untrusted bytes or bypass account permissions.

IMPLEMENTATION REQUIREMENTS

1. Create authenticated upload-intent, transfer-completion and status endpoints in services/api. Implement the minimum approved session/role boundary now if absent; full enrollment/recovery belongs to Goal 19. Development principals must be explicit fixtures that cannot enable anonymous production access.
2. Enforce proposed beta limits from configuration: at most 100 MB input and 60 seconds supported duration, plus per-account allowances and a separate processed-output budget. Record reservations atomically; reject concurrent quota bypass. Duration is verified by a bounded worker, not trusted client metadata.
3. Store uploads under generated names in private quarantine, using the approved local/object-storage adapter and narrowly scoped transfer authorization. Validate size, ownership, actual structure and completion digest. Do not trust filenames, extensions or claimed MIME types.
4. Use an outbox/job record for processing requests, stable idempotency keys and resumable status transitions. Clean abandoned/stale uploads and release reservations exactly once. Keep processing and publication separate.
5. Protect completion, status, cancellation and metadata access with object-level authorization. Record creator intent without private data on-chain. Add rate limits, structured safe errors and denied cross-account tests; never hard-code a user ID as authorization.

ACCEPTANCE CHECKS

Upload a permitted fixture and inspect its quarantine/job state. Test malformed input, oversized bodies, false completion, duplicate requests, concurrent quotas, interrupted transfer and expired authorization.

Attempt reads/deletes/completion for another account and verify denial. Restart the API/queue and demonstrate no double job or leaked quota.

Confirm quarantine files have no public playback path and an upload alone cannot become PUBLISHED.

REQUIRED HANDOFF

Deliver real upload APIs, safe fixture credentials, lifecycle tests and documented integration points. Keep public sign-up and public beta launch disabled.

GOAL 14 / 30

Isolated video processing workers

GPT-6.1 Sol	EFFORT: HIGH

Prerequisites: Goal 13; approved media/package formats and runnable worker environment.

Blueprint reference: pp. 5, 14, 16, 19 [B01]. Settings are selected in the client.

COPY PROMPT

Complete Goal 14. Read AGENTS.md, docs/progress.md and the approved specifications for this goal. Verify the target branch, base commit and prerequisite evidence. Execute only this goal; apply the common contract.

Build bounded FFmpeg processing that produces checked playback assets without exposing validators or application secrets.

IMPLEMENTATION REQUIREMENTS

1. Implement workers/media jobs using Celery/RabbitMQ or the approved queue adapter. Probe the actual input, enforce duration/dimensions/resource budgets and normalize orientation. Produce 480p/720p outputs when appropriate without upscaling, plus thumbnails and reviewed/user-provided captions.
2. Execute FFmpeg with argument arrays, generated paths and narrowly allowed protocols. Use an unprivileged isolated runtime, disposable workspace, bounded CPU/memory/disk/wall time and no unnecessary outbound network. Do not run untrusted media beside signer keys.
3. Create versioned manifests with segment/rendition metadata, hashes and byte totals. Validate outputs, synchronization and allowed formats. A new encoding creates a new package version; do not require separate encoders to emit identical bytes for consensus.
4. Make processing retry-safe with staging paths, atomic ready-for-review transitions and cleanup on cancel/crash. Reject excessive output growth. Preserve failure reasons without publishing stack traces, paths or secrets to users.
5. Record the exact FFmpeg build, codec configuration and licence notices. Where runtime isolation cannot be exercised, report that security boundary as unverified and retain an external test profile.

ACCEPTANCE CHECKS

Process valid portrait, rotated, low-resolution and audio/video fixtures; test malformed, truncated and over-budget inputs. Verify captions, duration, hashes and total bytes.

Kill a worker mid-job, restart and confirm one accepted package without leaked files/quota or accidental publication. Check network/process/resource restrictions directly where permitted.

Measure processing time on declared hardware; the proposed 120-second target for a 60-second input is a test gate, not an assumed result.

REQUIRED HANDOFF

Provide runnable workers, small permitted fixtures, sandbox evidence and build/licence records. Hand off verified package assets to delivery and provider services.

GOAL 15 / 30

Media manifests and HTTPS playback

GPT-6.1 Sol	EFFORT: HIGH

Prerequisites: Goal 14; package contract, origin storage and authorized content state.

Blueprint reference: pp. 6, 14, 16, 18, 26 [B01]. Settings are selected in the client.

COPY PROMPT

Complete Goal 15. Read AGENTS.md, docs/progress.md and the approved specifications for this goal. Verify the target branch, base commit and prerequisite evidence. Execute only this goal; apply the common contract.

Implement actual video delivery with validated manifests, host failover and accurate playback/integrity claims.

IMPLEMENTATION REQUIREMENTS

1. Create a versioned media-serving adapter and HLS master/rendition playlists over HTTPS for nonlocal deployments. Keep private/quarantined content inaccessible. Use controlled relative paths, safe content types, range handling where needed and narrowly configured cross-origin policy.
2. Return candidate hosts, immutable package identity and current publication/removal state from a typed API. Begin with the origin/test host adapter; Goal 16 supplies independent providers. Validate manifests and received replicas against hashes before advertising them.
3. Implement a test player or playback harness using the selected browser/HLS support. Add bounded retries and alternate-host selection. Do not claim every native HLS segment was client-verified unless the actual fetch/decode path supports and tests it.
4. Bound prefetch to a small approved window and cancel obsolete requests. Track startup, rebuffer time, playback errors and delivered bytes with privacy-preserving aggregates; distinguish an HTTP response from successful rendering.
5. Define controlled-cache invalidation and removal behaviour. A chain outage may allow approved playback within existing real-time terms with stale status, but cannot justify a new uncommitted hosting promise.

ACCEPTANCE CHECKS

Play real processed media and validate playlist/segment references, orientation, captions and fallback quality. Corrupt a manifest/segment and verify detection at the claimed verification layer.

Test host failure, 404s, slow responses, abandoned scroll requests and revoked content. Ensure no endless retry or unrestricted download of queued videos.

Run headless/browser checks only where supported; otherwise provide CI/manual checks and mark rendering unverified rather than inferring it from curl.

REQUIRED HANDOFF

Deliver delivery code, a real playback demonstration and an integrity-coverage statement. Leave optional P2P, TURN and public IPFS disabled.

GOAL 16 / 30

Storage-provider agent

GPT-6.1 Sol	EFFORT: HIGH

Prerequisites: Goals 07-09, 11-15; provider service and signing contracts.

Blueprint reference: pp. 9, 13-16, 19, 22 [B01]. Settings are selected in the client.

COPY PROMPT

Complete Goal 16. Read AGENTS.md, docs/progress.md and the approved specifications for this goal. Verify the target branch, base commit and prerequisite evidence. Execute only this goal; apply the common contract.

Implement the independently installable Go provider agent with explicit hosting consent and bounded local resources.

IMPLEMENTATION REQUIREMENTS

1. Create agents/provider with enrollment, signed capability/endpoint manifests, offer handling, authenticated package transfer, inventory, HTTPS serving and health/status endpoints. Use approved service identities and per-workspace test keys; keep real provider keys outside the development agent.
2. Check operator policy, actual available disk and configured quotas before staging data. Validate every transferred package/segment digest and byte bound, then sign exact accepted terms. Submit or forward acceptance through the approved provider-signing path; a service cannot manufacture it.
3. Persist local offer/transfer/lease state and reconcile committed obligations after restart. Separate STAGED, ACCEPTANCE_PENDING, COMMITTED and SERVING readiness, following the approved state machine. Apply real-time deadlines and expose uncertainty during chain stalls.
4. Defend all remote fetches against SSRF, path traversal, unsafe redirects and unbounded responses. Permit private devnet destinations only through explicit operator configuration, never by disabling checks globally.
5. Implement inventory/audit responses, bandwidth/storage limits, key-rotation hooks, local removal enforcement and the initial drain mode. Do not count several identities on one machine as independent ownership or physical redundancy.

ACCEPTANCE CHECKS

Install two isolated test agents, transfer real packages and verify each acceptance is bound to its own key and exact terms. Reject wrong signatures, altered bytes, unapproved offers, insufficient disk and repeated requests.

Restart after partial transfer and after committed acceptance; verify correct inventory, cleanup and capacity. Test a chain stall, lease deadline and local withdrawal.

Attempt unauthorized admin access and malicious endpoint/path fixtures inside the permitted test environment.

REQUIRED HANDOFF

Return a documented agent install, protocol conformance tests and real serving evidence. Identify which independence tests still require another person or machine.

GOAL 17 / 30

Replication, monitoring and repair

GPT-6.1 Sol	EFFORT: HIGH

Prerequisites: Goal 16 and canonical lease read models from Goal 12.

Blueprint reference: pp. 5-6, 13-15, 22, 26 [B01]. Settings are selected in the client.

COPY PROMPT

Complete Goal 17. Read AGENTS.md, docs/progress.md and the approved specifications for this goal. Verify the target branch, base commit and prerequisite evidence. Execute only this goal; apply the common contract.

Coordinate publication and repair across designated hosts without treating promises or node counts as durable storage proof.

IMPLEMENTATION REQUIREMENTS

1. Implement an off-chain scheduler using admitted operator/failure-domain records, compatible policies, declared capacity and measured reachability. Require explicit provider acceptance; do not let the scheduler sign for providers or perform external work inside consensus.
2. For the proposed beta, require community approval, verified package transfer and three active operator-distinct commitments before publication. Keep observed reachability, evidence mode and committed lease state separate. Local simulated operator labels must be disclosed as simulation.
3. Build bounded retrieval monitoring and an AT_RISK availability view. Select replacements, transfer from a surviving replica, verify bytes and confirm the new commitment before a planned departure removes needed data. Use idempotent repair jobs and avoid retry storms.
4. Implement renewal notices, deadlines, provider draining and orderly handoff. Respect withdrawal and removal policy; repair must not resurrect withdrawn content. During chain stalls, queue new final commitments and label any temporary off-chain service accurately.
5. Prepare three active test providers plus one spare or a safely provisioned replacement for the repair test. Track origin backups separately from counted replicas and report the actual physical failure domains.

ACCEPTANCE CHECKS

Publish a permitted package, disconnect creator/origin and one provider, retrieve from surviving hosts, and restore the replica target with the spare. Verify repaired bytes and chain accounting.

Test simultaneous failures, a dishonest receipt, low capacity, stale evidence, renewal expiry and duplicate repair work. Document inability to recover when all reachable copies are lost.

Drain a provider without prematurely deleting the last needed copy; remove content during repair and ensure distribution stops on cooperating services.

REQUIRED HANDOFF

Produce failure/repair traces, scheduler policy and honest availability status. Mark independent-operator verification separately from single-host topology tests.

GOAL 18 / 30

End-to-end network milestone

GPT-6.1 Sol	EFFORT: HIGH

Prerequisites: Goals 05-17 verified as far as their required runtimes allow.

Blueprint reference: pp. 3-4, 13-16, 25-28 [B01]. Settings are selected in the client.

COPY PROMPT

Complete Goal 18. Read AGENTS.md, docs/progress.md and the approved specifications for this goal. Verify the target branch, base commit and prerequisite evidence. Execute only this goal; apply the common contract.

Integrate a real upload-to-playback path with committed hosting records and demonstrable survival of host loss.

IMPLEMENTATION REQUIREMENTS

1. Connect the API, database/outbox, media worker, broadcaster, indexer, provider agents, scheduler and delivery layer. Use real services and the four-validator chain. Restrict access to explicit invited development principals until Goal 19 completes the account system.
2. Create a scripted scenario using a small permitted video: upload, process, obtain moderation approval, transfer verified packages, accept provider leases, confirm committed execution and publish. Expose each intermediate state rather than collapsing it into a success flag.
3. Extend the scenario to origin/uploader disconnection, one provider failure, replica repair and recovery of a restarted service. Save the exact chain ID, heights, public transaction IDs, package digest, byte checks and test timestamps.
4. Exercise loss of consensus quorum and verify existing permitted playback continues only under current real-time terms, while new commitments remain pending. Resume without resetting the chain or inventing accepted history.
5. Fix cross-service contract mismatches in small changes with updated fixtures and regression tests. Do not start social expansion until this milestone has real evidence; unavailable infrastructure remains a blocker, not a mock substitute.

ACCEPTANCE CHECKS

Run the complete scenario on a clean approved integration runner. Verify one final business effect after injected timeout/retry and no leaked quota or reservations.

Corrupt a package, reject an unauthorized provider acceptance and withdraw a test video from controlled services. Confirm UI/API status matches actual state.

Include a minimal independent client or protocol script that reads approved metadata and plays/retrieves media without private backend shortcuts.

REQUIRED HANDOFF

Deliver a repeatable demonstration and requirements-to-evidence report. Explicitly distinguish a working prototype from a reviewed public beta; list blockers before Goals 19-24.

GOAL 19 / 30

Accounts, recovery and data portability

GPT-6.1 Sol	EFFORT: HIGH

Prerequisites: Goal 18; approved custody, privacy and recovery decisions.

Blueprint reference: pp. 4-5, 17, 19-21 [B01]. Settings are selected in the client.

COPY PROMPT

Complete Goal 19. Read AGENTS.md, docs/progress.md and the approved specifications for this goal. Verify the target branch, base commit and prerequisite evidence. Execute only this goal; apply the common contract.

Replace development-only access with the approved account, session, recovery and signed-record system.

IMPLEMENTATION REQUIREMENTS

1. Implement invited enrollment, stable account IDs, session management, role/object authorization, revocation and abuse controls using a maintained authentication implementation. Add passkeys where the selected implementation supports them; do not invent authentication cryptography.
2. Keep application credentials separate from operator chain signing. Protect cookies/tokens, CSRF boundaries and recovery flows. Require strong confirmation for sensitive changes, rotate sessions and rate-limit enrollment/login/recovery. Remove any production-reachable development bypass.
3. Implement delegated host-signing custody only as approved and disclose that trust. Version public record envelopes, key references, sequence values and canonicalization; test key history and replay. Saved videos, preferences, recovery data and raw telemetry remain private/off-chain.
4. Provide account exports, verified import/migration between compatible test hosts, deletion initiation and a defined encrypted backup path. Preserve identity only to the extent the approved custody scheme supports it; do not claim full federation or guaranteed outside follower migration.
5. Apply retention and least-privilege access to account data. Exclude other people's private records and confidential moderation evidence from exports. Keep any unresolved legal/privacy launch decision visibly blocked.

ACCEPTANCE CHECKS

Test cross-account access, expired/reused recovery links or challenges, stolen/revoked sessions, duplicate enrollment, CSRF and privilege escalation within the test system.

Rotate keys and validate old/new signed-record handling with cross-language fixtures. Migrate a test account between two compatible hosts and verify permitted data integrity.

Delete a test account, deny future actions and verify controlled removal/retention behaviour without claiming all historical public records disappear.

REQUIRED HANDOFF

Deliver real user authentication and recovery evidence. Audit every earlier endpoint against the new identities and rerun the end-to-end scenario.

GOAL 20 / 30

Viewer web application

GPT-6.1 Sol	EFFORT: HIGH

Prerequisites: Goals 15, 18-19; current typed playback/account APIs.

Blueprint reference: pp. 4-5, 16, 18, 26 [B01]. Settings are selected in the client.

COPY PROMPT

Complete Goal 20. Read AGENTS.md, docs/progress.md and the approved specifications for this goal. Verify the target branch, base commit and prerequisite evidence. Execute only this goal; apply the common contract.

Build the responsive viewer experience against real APIs, with accessible playback and honest loading/error states.

IMPLEMENTATION REQUIREMENTS

1. Implement apps/web in React/TypeScript with Home, Discover, Activity and Profile navigation as appropriate to available endpoints. Deliver the vertical video player, public profiles, captions, mute, keyboard navigation, reduced motion and responsive layouts.
2. Connect playback to approved content metadata, candidate hosts and account sessions. Implement bounded prefetch, cancellation, network fallback, expired-session handling and clear unavailable/withdrawn/degraded states. No mandatory wallet or peer upload is required.
3. Use typed client adapters and consistent errors; treat titles, captions and comments as untrusted data. Avoid unsafe HTML rendering and leaking secrets or private preferences into URLs/telemetry.
4. Wire existing actions to real endpoints. For social/moderation features scheduled in Goals 22-23, use explicit unavailable/feature-gated states, not mock success. Keep the product an invited development build until reporting/blocking are complete.
5. Create component and accessibility tests plus browser-level flows. When interactive browser tools are absent, use permitted headless tests or a separate CI/manual validation path; screenshots or visual approval must never be fabricated.

ACCEPTANCE CHECKS

Watch actual processed videos, change quality, scroll rapidly, lose a host and recover. Verify no runaway downloads, repeated audio or access to withdrawn/quarantined content.

Test keyboard-only navigation, captions, reduced motion, mobile viewport sizes, expired login and API failures. Record browser/device details for visual tests.

Confirm unavailable features are visibly gated and no test fixture is presented as a real user or live network statistic.

REQUIRED HANDOFF

Provide the usable viewer build, real browser evidence where available and remaining feature gates. Do not claim visual verification from a successful web build alone.

GOAL 21 / 30

Creator publishing interface

GPT-6.1 Sol	EFFORT: HIGH

Prerequisites: Goals 13-20; real upload, account and media-status APIs.

Blueprint reference: pp. 4-5, 13-17, 26 [B01]. Settings are selected in the client.

COPY PROMPT

Complete Goal 21. Read AGENTS.md, docs/progress.md and the approved specifications for this goal. Verify the target branch, base commit and prerequisite evidence. Execute only this goal; apply the common contract.

Build creator workflows for publishing and managing real videos, with accurate processing, hosting and removal status.

IMPLEMENTATION REQUIREMENTS

1. Implement upload selection, bounded basic trimming, title/description, captions, thumbnail selection and accessible progress views. Use the existing server pipeline; trim requests must still satisfy server-side input, duration and processing budgets.
2. Display Uploading, Processing, Review pending, Replicating and Published separately. Show failures, stalled-chain pending state, availability degradation and hosting-renewal notices. Do not show Published from a local optimistic state before the approved conditions hold.
3. Provide cancel/retry, draft management, publication intent and removal requests. Require object ownership and confirmation for destructive actions. Do not allow changing media bytes under the same immutable content version.
4. Integrate account recovery/export links and transparent hosting terms/allowances. Keep creator analytics to implemented privacy-preserving aggregates; never invent views, revenue or storage guarantees.
5. Handle network interruption, expired upload authorization, excessive media size, rejected content and exhausted quota. Use typed API errors and keep repeated actions idempotent.

ACCEPTANCE CHECKS

Publish a permitted fixture end to end and inspect every displayed lifecycle state against backend and committed-chain evidence. Test creator/origin disconnection and later playback.

Try another account's edit/remove actions, illegal trim ranges, duplicate submissions, cancellation during processing and retry after a timeout. Verify correct reservations and no leaked files.

Withdraw a video and ensure the UI does not claim universal deletion; present pending participating-host actions accurately.

REQUIRED HANDOFF

Deliver a working creator flow with test evidence and any dependencies on Goal 23 moderation completion. Preserve the invited-development boundary.

GOAL 22 / 30

Social features, search and feeds

GPT-6.1 Sol	EFFORT: HIGH

Prerequisites: Goals 19-21; signed-record contract and approved privacy rules.

Blueprint reference: pp. 4, 17-18, 26 [B01]. Settings are selected in the client.

COPY PROMPT

Complete Goal 22. Read AGENTS.md, docs/progress.md and the approved specifications for this goal. Verify the target branch, base commit and prerequisite evidence. Execute only this goal; apply the common contract.

Implement the bounded social layer and understandable discovery without putting ordinary interactions on-chain.

IMPLEMENTATION REQUIREMENTS

1. Add follow/unfollow, likes, comments, private saves, in-app activity and Not interested actions with authorization, pagination, rate limits and retry-safe writes. Use stable account/content IDs and approved signed public records where specified.
2. Implement searchable permitted profiles/topics/videos and a chronological Following feed. Add a separate simple recommendation feed using candidates, explicit scoring factors and final filtering; no new ML training service is required.
3. Exclude unavailable, withdrawn, blocked and locally disallowed content. Limit repeated creators and preserve user feed preferences. Validate comment inputs, safe rendering and deletion propagation; keep exact abuse-control internals private where necessary.
4. Expose versioned discovery interfaces and rebuildable indexes so another compatible service can supply results. Do not claim ActivityPub/AT Protocol interoperability without an implemented adapter and conformance tests.
5. Wire the viewer/creator UI to the real endpoints and remove completed feature gates. Keep raw viewing histories, saves and private preferences out of public records, chain events and public analytics.

ACCEPTANCE CHECKS

Test follow graph permissions, duplicate likes, pagination under concurrent updates, comment moderation boundaries, private saves and notification access.

Rebuild search/feed indexes from permitted records and compare content coverage. Verify chronological order and filtering after withdrawal/blocking, including cached pages.

Use a small second client to request a feed and interpret stable content IDs. Report relevance tests as fixture results, not proof of superior recommendations.

REQUIRED HANDOFF

Deliver functional social interactions, discovery contracts and filtered-feed tests. Hand off moderation-sensitive endpoints and cache paths to Goal 23.

GOAL 23 / 30

Moderation, blocking and removal

GPT-6.1 Sol	EFFORT: HIGH

Prerequisites: Goals 17-22; approved community policy and retention decisions.

Blueprint reference: pp. 4-5, 14, 16-21, 26 [B01]. Settings are selected in the client.

COPY PROMPT

Complete Goal 23. Read AGENTS.md, docs/progress.md and the approved specifications for this goal. Verify the target branch, base commit and prerequisite evidence. Execute only this goal; apply the common contract.

Complete the safety workflows required before invited public use and make removal promises technically precise.

IMPLEMENTATION REQUIREMENTS

1. Implement reporting, account blocking, an administrator queue, decisions with reasons, appeals and escalation status. Enforce moderator permissions and audit administrative actions; sensitive complainant/evidence data stays access-controlled and off-chain.
2. Apply local policy consistently at upload admission, publication, feeds/search, comments, storage admission and clients. A committed transaction cannot override a community's right to stop distributing content on its services.
3. Implement removal as coordinated, retryable delisting, controlled-cache invalidation, participating-host deletion and lease cancellation/end according to approved terms. Stop repair/renewal from reviving withdrawn content. During a chain stall, local delisting must not wait for new finality.
4. Track each cooperating service's acknowledgement and unreachable hosts separately. Disclose independently retained copies and historical ledger records that cannot be erased by this operation. Preserve only records permitted by the approved retention policy.
5. Provide configurable policy/contact pages and operational runbooks with placeholders for real owners, not fabricated legal approval. Keep public launch blocked until policy, incident coverage, age rules and rights/privacy review are approved.

ACCEPTANCE CHECKS

Test unauthorized moderator actions, confidential-case access, spam reports, block interactions, appeals and account takedown. Verify all UI report/block actions have real backend effects.

Remove content while it is uploading, processing, replicating, being repaired and being played. Test offline providers and a stalled chain; report incomplete deletion honestly.

Measure the proposed five-minute controlled-service delisting gate with clocks, affected endpoints and cache evidence. An unresponsive independent host is not a successful deletion.

REQUIRED HANDOFF

Deliver safety workflows, an operator runbook and removal coverage results. Separate software completion from professional policy/legal approval.

GOAL 24 / 30

Operator dashboard and node onboarding

GPT-6.1 Sol	EFFORT: HIGH

Prerequisites: Goals 06-12, 16-17, 19, 23; stable operator APIs.

Blueprint reference: pp. 4, 12-15, 19-22 [B01]. Settings are selected in the client.

COPY PROMPT

Complete Goal 24. Read AGENTS.md, docs/progress.md and the approved specifications for this goal. Verify the target branch, base commit and prerequisite evidence. Execute only this goal; apply the common contract.

Build a real operator interface for bounded participation, governance visibility and orderly exit.

IMPLEMENTATION REQUIREMENTS

1. Implement role-specific onboarding for storage/delivery, processing, account/discovery and validators. Let operators choose roles independently. Document machine/network requirements as measured observations, not universal minimums.
2. Show actual disk allocation, leases, inventory, job queues, bandwidth counters, evidence mode, chain/index height, overdue work and software version. Distinguish configured, measured, pending and stale values; do not create decorative fake metrics.
3. Provide authorized capacity/policy changes, stop-new-offers controls, drain/handoff, renewal review and local endpoint disablement. Make dangerous actions explicit and auditable; the dashboard cannot bypass chain authorization or commit an unapproved governance action.
4. Display council proposals, approval state, activation timing and upgrade instructions without handling live consensus private keys in the browser. Keep public/read-only endpoints separate from signing/admin endpoints.
5. Expose local moderation/report queues and health diagnostics with least privilege. Show operator ownership/failure-domain declarations accurately and label single-machine demos as such.

ACCEPTANCE CHECKS

Onboard a fresh test provider and observe real package acceptance and delivery. Change limits, exhaust capacity, drain the provider and verify handoff/lease accounting.

Exercise viewer/provider/moderator/council role separation and deny cross-role actions. Confirm stale chain or index data is visible rather than silently presented as current.

Run browser tests and a manual independent-operator walkthrough where an operator is available; leave that evidence blocked when no independent person participated.

REQUIRED HANDOFF

Deliver the dashboard, tested onboarding and exit flows, and a role/permission map. Record operational tasks that still require an administrator.

GOAL 25 / 30

Observability, quotas and usage controls

GPT-6.1 Sol	EFFORT: HIGH

Prerequisites: Goals 18-24; actual service metrics and account boundaries.

Blueprint reference: pp. 19, 22-24, 26 [B01]. Settings are selected in the client.

COPY PROMPT

Complete Goal 25. Read AGENTS.md, docs/progress.md and the approved specifications for this goal. Verify the target branch, base commit and prerequisite evidence. Execute only this goal; apply the common contract.

Make resource use, failures and budget exposure measurable without collecting unnecessary personal data.

IMPLEMENTATION REQUIREMENTS

1. Instrument API, jobs, broadcaster, indexer, chain, providers and playback using consistent correlation IDs that do not expose private identities. Add health/readiness probes and logs with secret/personal-data redaction and bounded retention.
2. Measure commit age, index lag, processing age, reachable replicas, expiry/repair backlog, queue retries, delivered bytes, storage and playback failures. Separate user payload, replication, repair, prefetch and backups rather than double-counting or inventing savings.
3. Enforce admission limits for uploads, processing concurrency, output bytes, storage reservations, audit batches and retries. Add hard stop/queue controls when an approved budget or capacity threshold is reached. Alerts alone must not be described as a spend cap.
4. Create operational dashboards and alert/runbook mappings with named-role placeholders awaiting real assignment. Use aggregate playback telemetry and account/privacy controls; do not put watch events on-chain.
5. Provide a cost worksheet that separates measured units from operator-supplied rates and currency. Keep total network resources distinct from founder expenses and volunteer contributions. Do not forecast a monthly bill without real inputs.

ACCEPTANCE CHECKS

Inject slow processing, low disk, stale commits, failed repairs and repeated uploads; verify detection, safe intake controls and recovery. Ensure health checks do not report readiness when a critical dependency is unusable.

Check redaction with synthetic secret canaries and access controls on metrics/logs. Test quota races and prove repeated requests cannot bypass hard limits.

Compare measured byte/storage counters against known fixtures and document any unmeasured traffic classes.

REQUIRED HANDOFF

Deliver metrics, resource controls, runbooks and an auditable usage report. Record budget-owner and operational-coverage decisions still needed for release.

GOAL 26 / 30

Backups, restoration and signer safety

GPT-6.1 Sol	EFFORT: HIGH

Prerequisites: Goals 10, 18-25; approved retention, key custody and recovery policy.

Blueprint reference: pp. 12, 17, 20, 22, 26 [B01]. Settings are selected in the client.

COPY PROMPT

Complete Goal 26. Read AGENTS.md, docs/progress.md and the approved specifications for this goal. Verify the target branch, base commit and prerequisite evidence. Execute only this goal; apply the common contract.

Implement and rehearse recovery without conflating live replication with backup or risking duplicate validator signing.

IMPLEMENTATION REQUIREMENTS

1. Create documented backups for application databases, signed records, relevant configuration, critical media and index reconstruction. Encrypt sensitive backups, constrain access and define retention and restore order. Exclude live secrets from ordinary CI artifacts.
2. Implement a disposable restore harness on clean services/hosts with the original application and media origin unavailable. Rebuild chain projections from a trusted checkpoint or supported canonical state and verify application/media references.
3. Document chain-state and signer-safety metadata handling for the selected implementation. Require the old signer to be stopped and verified before replacement. Never run duplicate consensus identities, roll signer safety backwards or treat a stale snapshot as permission to sign conflicting state.
4. Rehearse service-key rotation, account recovery, corrupted-backup detection, missing media and partial restore. Explain migration/version compatibility and which historical data cannot be reconstructed from current replicas alone.
5. Document orderly end-of-service exports, stopped admissions, lease handoff and controlled deletion. Production restore or key rotation requires explicit operator approval; use only test keys here.

ACCEPTANCE CHECKS

Restore the database, media and index onto a clean approved runner and repeat publication/playback/removal checks. Compare digests and authorization state before/after.

Measure actual recovery time and recoverable data age for the test; label them observed RTO/RPO, not service guarantees.

Attempt corrupted, wrong-version and incomplete restore fixtures. Verify duplicate-live-signer prevention in the controlled test and record any unverifiable external steps.

REQUIRED HANDOFF

Deliver tested backup/restore commands, compatibility notes and signer-safety evidence. List manual approvals and independent review required before live recovery.

GOAL 27 / 30

Packaging, installation and release preparation

GPT-6.1 Sol	EFFORT: HIGH

Prerequisites: Goals 03-26; approved licence decisions and tested recovery paths.

Blueprint reference: pp. 7, 19, 21-22, 25-28 [B01]. Settings are selected in the client.

COPY PROMPT

Complete Goal 27. Read AGENTS.md, docs/progress.md and the approved specifications for this goal. Verify the target branch, base commit and prerequisite evidence. Execute only this goal; apply the common contract.

Prepare reproducible installable artifacts and release documentation without automatically publishing or deploying them.

IMPLEMENTATION REQUIREMENTS

1. Create reference deployment profiles for the web/API, workers, provider agents, query/index services and validators. Document persistent volumes, TLS, network ports, secrets, health checks, resource observations and upgrades. Keep untrusted media processing separate from signer material.
2. Build versioned binaries/images and an installation/configuration flow with secure defaults. Pin image/dependency versions, generate SBOMs, retain third-party notices and record FFmpeg codec/build obligations. Resolve licence blockers before labelling artifacts releasable.
3. Provide generated-on-install development fixtures only; no shared private keys or default production credentials. Create verification commands for artifact digests and a signing procedure. Test signing keys must be visibly nonproduction; actual release signing remains an owner action.
4. Prepare README, operator quickstart, protocol/API reference, CONTRIBUTING, SECURITY, governance notes and migration/recovery instructions. Publish known limits and trust boundaries. Do not claim interoperability, security audits or independent operation not demonstrated.
5. Configure a safe release-candidate workflow with minimal permissions and approval gates. Do not push tags, publish packages, open network ports, register domains or deploy paid services unless explicitly authorized.

ACCEPTANCE CHECKS

Build from a clean pinned checkout, install on a fresh approved test runner and run the documented smoke scenario without undocumented steps. Compare build inputs and explain any non-reproducible outputs.

Inspect bundles/images for secrets, excessive privileges, missing notices and unpinned downloads. Test uninstall/cleanup without deleting unrelated operator data.

Rehearse an upgrade and rollback only where the migration design permits it.

REQUIRED HANDOFF

Produce local release-candidate artifacts, checksums and install evidence. Mark the package NOT RELEASED until Goals 28-30 and human launch gates pass.

GOAL 28 / 30

Performance, load and capacity evidence

GPT-6.1 Sol	EFFORT: HIGH

Prerequisites: Goal 27; approved integration/load runner and resource budget.

Blueprint reference: pp. 16, 22-24, 26 [B01]. Settings are selected in the client.

COPY PROMPT

Complete Goal 28. Read AGENTS.md, docs/progress.md and the approved specifications for this goal. Verify the target branch, base commit and prerequisite evidence. Execute only this goal; apply the common contract.

Measure the actual beta system against its proposed targets without extrapolating unsupported scale claims.

IMPLEMENTATION REQUIREMENTS

1. Create a reproducible load plan with pinned revision, hardware, OS, network settings, video corpus, cache state, concurrency and duration. Use permitted synthetic/owned media, disposable accounts and a bounded spend/resource allowance.
2. Measure the blueprint design point of 100 simultaneous viewers and 10 upload sessions. Under a controlled 20 Mb/s downlink and 50 ms RTT, evaluate p95 playback startup at or below 3 seconds and rebuffer ratio at or below 1 percent. Define exactly how the network limit is applied and every metric denominator.
3. Test a 60-second supported upload reaching ready-for-review within the proposed 120-second target at declared worker concurrency/hardware. Separate processing, moderation, replication and total publication time.
4. Measure chain transaction bytes/gas, state growth, commit behaviour, index lag, repair time and service-resource saturation. Add overload, slow-provider and host-loss scenarios with bounded duration; ordinary tests must not be confused with attacks on third-party systems.
5. Fix evidenced bottlenecks without weakening integrity, quotas or acceptance thresholds. Rerun comparable tests after each material change. Prefer measured changes over speculative microservices or complex recommendation infrastructure.

ACCEPTANCE CHECKS

Save raw results and calculations with the tested code revision. Report target, observed value, sample size and pass/fail/blocked status for every gate.

Cross-check client rendering measurements against server telemetry; a successful segment download alone is not playback success.

If the cloud workspace cannot run shaped-network/browser/full-load tests, run them on an authorized fuller runner or mark them blocked. Do not invent numbers or treat short smoke tests as capacity proof.

REQUIRED HANDOFF

Deliver benchmark evidence, resource sizing observations and remaining limits. No claim of TikTok-scale capacity or an actual operating budget follows from these beta tests.

GOAL 29 / 30

Adversarial review and fault testing

GPT-6 Astra	EFFORT: EXTRA HIGH

Prerequisites: Goals 01-28; exact candidate revision and evidence inventory.

Blueprint reference: pp. 10, 12-15, 19, 26-28 [B01]. Settings are selected in the client.

COPY PROMPT

Complete Goal 29. Read AGENTS.md, docs/progress.md and the approved specifications for this goal. Verify the target branch, base commit and prerequisite evidence. Execute only this goal; apply the common contract.

Independently challenge the candidate implementation and its tests, then fix verified defects with regression evidence.

IMPLEMENTATION REQUIREMENTS

1. Begin by inspecting code, approved invariants and trust boundaries before relying on earlier completion summaries. Review chain authorization, validator lifecycle, lease accounting, replay/idempotency, custody/recovery, API object permissions, media sandboxing, SSRF and supply-chain workflows.
2. Use unit/property/fuzz/contract tests plus controlled fault injection on owned test systems. Exercise dishonest providers/auditors, repeated IDs, stale evidence, outsourcing assumptions, partial writes, quorum loss, corrupted packages, blocked removals and compromised test service keys.
3. Inspect tests for mocks in end-to-end paths, swallowed errors, skipped cases, permissive assertions, leaked secrets and unjustified expected failures. Search for stubs, fake metrics, TODOs in critical paths and stale feature flags.
4. For each finding, record severity rationale, affected code, preconditions, a safe local reproduction, impact and fix proposal. Implement bounded fixes with regression tests, then rerun affected integration suites at the new candidate revision.
5. Separate model-assisted review from an independent human security audit. Do not claim unrestricted permissionless security, cryptographic proof guarantees or protection from all hostile operators. Preserve unresolved blockers and required outside reviews.

ACCEPTANCE CHECKS

Confirm rejected chain operations leave state unchanged, replay hashes match at the same height, and partial failures do not create false commitment or duplicate reservations.

Rerun host-loss/repair, chain-stall and deletion-during-repair scenarios. Verify failures remain visible to clients and operators.

Produce a finding-by-finding closure matrix; a finding is closed only with code evidence and an executed regression test or a clearly approved scope decision.

REQUIRED HANDOFF

Return the reviewed revision, fixed/open findings and remaining external review needs. Prepare an evidence-based release assessment, not a generic security endorsement.

GOAL 30 / 30

Final acceptance and independent installation

GPT-6 Astra	EFFORT: EXTRA HIGH

Prerequisites: Goal 29; all mandatory checks resolved or explicitly recorded as release blockers.

Blueprint reference: pp. 3-4, 22, 25-28 [B01]. Settings are selected in the client.

COPY PROMPT

Complete Goal 30. Read AGENTS.md, docs/progress.md and the approved specifications for this goal. Verify the target branch, base commit and prerequisite evidence. Execute only this goal; apply the common contract.

Determine whether the permissioned beta meets its specification, with reproducible evidence and an honest release decision.

IMPLEMENTATION REQUIREMENTS

1. Freeze a candidate revision and audit every requirement-to-test link. Reproduce builds, schema generation, unit/contract/integration checks, migration, backup/restore and relevant performance results from clean permitted environments. A code change invalidates affected prior evidence until rerun.
2. Run the full success demonstration: a creator publishes permitted media; real hosts accept exact terms; the chain commits obligations; the origin disconnects; another client watches; one provider leaves; repair restores the target; controlled services remove a test item.
3. Give the documented installer to a genuinely separate operator on separate infrastructure where available. Ask them to install and serve the agreed catalogue without private setup instructions. If no such operator participated, label this as an installation rehearsal and leave independence unverified.
4. Use a second minimal client to exercise the published interfaces and verify there is no mandatory private backend shortcut. Check source/licence notices, artifact integrity, actual governance ownership, operator responsibilities, recovery and incident coverage.
5. Produce a release decision with distinct categories: implemented prototype, beta candidate, or beta approved after required human sign-offs. List all blockers, deferred extensions and residual risks. No automatic public deployment, release publication, token launch or paid provisioning.

ACCEPTANCE CHECKS

Every mandatory acceptance gate has executed evidence tied to the candidate or is visibly blocking release. Test output must show failures/skips, not only a green summary.

Obtain actual owner sign-off for moderation coverage, privacy/rights policy, funding/quotas, operator participation and security review; never fabricate names or approvals.

Document rollback/recovery constraints and a safe first-community launch plan bounded by observed capacity.

REQUIRED HANDOFF

Deliver docs/release-readiness.md, the final evidence index and a concise operator handoff. State exactly what is usable and what remains unverified; completing 30 prompts is not itself approval.

WHEN A TASK NEEDS ANOTHER PASS

Continuation, repair and review

These are reusable follow-ups, not extra numbered implementation goals. Replace NN with the current goal. Reuse the same branch/task for a repair, or create a separate review task against a clearly identified diff.

Continue unfinished work

Use the current goal's model and effort, unless a specific blocker justifies a different setting.

Continue Goal NN on the intended repository and branch. Read AGENTS.md, docs/progress.md and docs/evidence/goal-NN.md. Inspect the current diff and tested commit before changing anything. Reproduce the smallest outstanding failure, preserve unrelated work, implement a bounded fix and rerun the affected tests. Do not repeat completed tasks or expand scope. Report actual commands, results, remaining blockers and the new checkpoint. Never treat an unavailable test as passed.

Diagnose a failed acceptance check

Begin with the assigned model. Escalate a demonstrated protocol/cross-system ambiguity rather than automatically rerunning everything at higher effort.

The acceptance check for Goal NN failed. Do not change its threshold or remove the assertion. First determine whether the failure is in implementation, test setup, the test itself, or an unresolved specification. Show the relevant evidence. If the test is demonstrably wrong, propose and justify a correction before changing it. Otherwise fix the implementation, add a regression case and rerun the relevant suite. Record all remaining failed or unrun checks.

Review without rewriting the project

Suggested: Astra High for security-critical diffs; Extra High for the final cross-system review. An additional model pass is not an independent human audit.

Review the diff for Goal NN without accepting the author's completion claim. Compare it with approved requirements, public interfaces and acceptance tests. Inspect permissions, state transitions, negative cases, retries, migrations, secrets and test shortcuts. Report concrete findings with file/line locations and a safe reproduction. Do not rewrite unrelated code or merge the PR. A model review does not replace an independent security audit.

Keep context and token use bounded

Maintain concise progress, decisions and evidence files. Read only the relevant code and source sections after the initial specification. Avoid sending the entire blueprint and all logs on every turn. Preserve a minimal failing case, summarize long traces and link to raw artifacts.

After a verified dependency change, update the reusable environment deliberately and check the base revision in the next task. Do not assume an older task inherited another task's uncommitted changes. This workflow is a recommendation; no token-budget guarantee is made.

When blocked: state the missing capability or decision, what was still tested, and the exact next step. Do not repeatedly regenerate code when the real blocker is unavailable infrastructure or an unapproved protocol rule.

EVIDENCE BEFORE RELEASE

Acceptance and handoff checklist

This is the beta release rubric from the blueprint, expanded into explicit evidence requirements. All numerical values are proposed gates, not achieved benchmarks [B01, p. 26].

Chain correctness	Same-height replay hashes agree. Unauthorized, forged and replayed messages fail without mutating state. Record quorum-loss and recovery behaviour.
Hosting resilience	Three designated copies; origin and one provider removed; surviving bytes retrieved and repair completed. Separately disclose actual operator/failure-domain independence.
Integrity and retries	Corrupted packages are rejected at the stated verification layer. Repeated operations, timeouts and restarts leave one intended effect and accurate capacity.
Playback and processing	Measure p95 startup <= 3 s and rebuffer <= 1% under the declared 20 Mb/s, 50 ms RTT test; a 60 s input targets <= 120 s to review-ready on declared hardware.
Load and recovery	Measure the 100-viewer / 10-upload design point. Restore database, media and index on clean hosts; document observed recovery time/data age and signer safety.
Safety and removal	Real report, block, appeal and removal flows; measure controlled-service delisting <= 5 min. Record unreachable services and independent-copy limitations.
Independent use	A separate operator installs from public instructions and a second client uses documented interfaces. Otherwise label the exercise a rehearsal, not independence.
Human launch gates	Actual owners approve governance, security review, moderation coverage, privacy/rights policy, quotas/funding and recovery. Missing approvals remain blockers.

Required evidence record

For every goal: requirement IDs; code revision; environment and dependency versions; exact commands; exit codes and pass/fail/skip counts; public test identifiers or digests; raw artifact locations; unresolved findings; and reviewer/owner decisions. Never include secrets or personal case evidence in the public record.

Final status language

Use implemented prototype, beta candidate, or approved beta only as justified. Unrun tests are unverified; missing security or policy approvals block public release. Thirty prompts are a plan, not an audit, uptime commitment or commercial forecast.

VERIFICATION AND ATTRIBUTION

Sources and interpretation

Documentation checked 13 October 2026. Recheck available model settings, environment capabilities, framework versions and licences during implementation. The references support factual tool/project context; goal allocations and instructions are original recommendations.

B01 Project blueprint, version 1.0, 12 October 2026

Source: the 33-page PDF supplied in this conversation. Page references on each goal identify the originating requirements. This is a proposed architecture, not evidence of completed software.

O01 OpenAI: Codex Cloud

https://learn.chatgpt.com/docs/cloud

GitHub-connected cloud setup, task execution, review and pull-request workflow.

O02 OpenAI: Cloud environments

https://learn.chatgpt.com/docs/environments/cloud-environments

Current setup/start fields, saved-state boundaries, environment controls, runtime resources and limitations. Distinct from the legacy environment guide.

O03 OpenAI: Models

https://learn.chatgpt.com/docs/models

Model identifiers, reasoning-effort controls and account/client availability. Goal-by-goal allocations are this document's recommendations, not vendor benchmarks.

O04 OpenAI: Custom instructions with AGENTS.md

https://learn.chatgpt.com/docs/agent-configuration/agents-md

Repository-level instructions for coding agents.

G01 GitHub: GitHub-hosted runners

https://docs.github.com/en/actions/concepts/runners/github-hosted-runners

Hosted workflow runners and configurable execution environments. Actual runner software and capacity must be checked.

G02 GitHub: Secure use reference

https://docs.github.com/en/actions/reference/security/secure-use

Least-privilege workflow credentials, dependency pinning/locking, and untrusted-code precautions.

How to interpret this pack

The blueprint and this playbook are specifications to implement, not runnable software. Exact SDK APIs and dependency versions must be checked against official sources when pinned. No new consensus algorithm or cryptographic storage-proof construction is supplied. Model/effort choices are not a guarantee of correctness, completion time or token consumption.

I cannot confirm that the complete development topology fits your current Codex Cloud workspace, that all acceptance targets will pass, or that the resulting public network will be secure or economical without implementation, measurement and review.

The GitHub companion pack includes the original blueprint and its page-labelled extracted text. Use the PDF to resolve any ambiguity in flattened text, tables or diagrams. The pack does not create a GitHub repository or configure your account.